TECHNICAL DOCUMENTATION

Developer Portfolio Website

How it was designed, developed and implemented: from file structure to code

Abdul Rauf Kaka

Frontend focused Full Stack Developer

arauf9229@gmail.com

October 31, 2026

React 19 | Vite | Tailwind CSS | GSAP

Table of Contents

Table of Contents 2

1. Project Overview 4

Goal 4

Analysing the reference website 4

Implemented features 4

2. Technology Stack and Libraries 6

3. Project Setup and Development Workflow 7

Step 1: Scaffold the project with Vite 7

Step 2: Install dependencies 7

Step 3: Clean the template 7

Step 4: Build features, then verify 7

Available npm scripts 7

4. Folder and File Structure 8

Why this structure? 8

5. Architecture and How It Works 10

Component tree 10

Data flow 10

Load sequence and the "ready" flag 10

Smooth scrolling and scroll animations 10

Routing 11

Animation lifecycle with useGSAP 11

6. Design System and Responsive Strategy 12

Design tokens 12

Responsive approach 12

Accessibility 13

7. Screenshots 14

8. Source Code Walkthrough 16

Configuration and Entry Point 16

package.json 16

vite.config.js 16

index.html 17

public/favicon.svg 17

src/main.jsx . . . 17
src/index.css . . . 18
src/App.jsx . . . 19
Content Data . . . 20
src/data/portfolio.js . . . 20
Hooks . . . 22
src/hooks/useSmoothScroll.js . . . 22
Layout Components . . . 23
src/components/layout/Preloader.jsx . . . 23
src/components/layout/ParticleBackground.jsx . . . 24
src/components/layout/ScrollProgress.jsx . . . 26
src/components/layout/Navbar.jsx . . . 26
src/components/layout/StickyEmail.jsx . . . 28
src/components/layout/Footer.jsx . . . 29
Reusable UI Components . . . 29
src/components/ui/Button.jsx . . . 29
src/components/ui/SectionTitle.jsx . . . 30
src/components/ui/ProjectThumbnail.jsx . . . 30
Home Page Sections . . . 31
src/sections/Hero.jsx . . . 31
src/sections/AboutMe.jsx . . . 33
src/sections/Skills.jsx . . . 34
src/sections/Projects.jsx . . . 35
Pages . . . 37
src/pages/Home.jsx . . . 37
src/pages/ProjectDetails.jsx . . . 37
src/pages/NotFound.jsx . . . 39

9. Customising and Deploying . . . 40

Updating content . . . 40
Changing the look . . . 40
Building and deploying . . . 40

10. Testing and Verification . . . 40

1. Project Overview

This document explains, end to end, how the personal portfolio website of **Abdul Rauf Kaka** was designed, built and implemented. It covers the goals of the project, the technologies chosen, how the project was set up, the complete folder structure, the architecture and design system, every source file with its full code and an explanation, and finally how to customise and deploy the site.

Goal

Build a modern, animated, fully responsive developer portfolio in **React.js**, scaffolded with **Vite**, modelled on the look and behaviour of the reference site https://www.tajmirul.site/, and filled with Abdul Rauf's own bio, tech stack and projects.

Analysing the reference website

Before writing any code, the reference site was opened in a browser and inspected. Its content structure, computed colours, fonts and interactions were recorded so they could be reproduced:

- **Theme:** dark background #212121, light text #DEDEDE, bright green accent (HSL 140 100% 47%, roughly #00F05A) and a cyan secondary colour.
- **Typography:** the condensed display font **Anton** for big headings and **Roboto Flex** for body text.
- **Preloader:** the owner's name animates in on a black screen before the page is revealed.
- **Background:** a field of small, slowly drifting white "stars".
- **Navigation:** a hamburger button in the top-right corner that slides open a panel with Menu, Social and Get in touch columns.
- **Sticky email:** the email address written vertically on the left edge of the screen.
- **Sections:** hero ("FULL-STACK DEVELOPER" + intro + Let's Talk button + stats), an About section with a large statement, My Stack with icons, Experience, Selected Projects (numbered list with a hover image preview), project detail pages, and a contact footer.
- **Motion:** smooth scrolling and scroll-triggered animations throughout.

Implemented features

Feature	How it is implemented
Name preloader	GSAP timeline animates each letter of "ABDUL RAUF" in and out, then slides the overlay away.
Starfield background	A full-screen HTML5 canvas draws ~200 twinkling particles with requestAnimationFrame.
Smooth scrolling	Lenis drives the scroll; it is synchronised with GSAP ScrollTrigger on every tick.
Scroll progress bar	A 3px gradient bar at the top whose scaleX follows the scroll percentage.

Feature	How it is implemented
Hamburger menu	Fixed round button toggles a slide-in panel; Escape, backdrop click and links close it.
Hero section	Masked line-by-line title reveal, fading intro, stats, and a scroll-scrubbed fade-out.
About section	Statement words brighten one by one as you scroll (scrubbed ScrollTrigger).
My Stack	Brand icons from react-icons grouped by category, staggered in on scroll.
Selected Projects	Numbered list; on desktop a preview card follows the cursor (gsap.quickTo).
Project detail pages	React Router route /projects/:slug with overview, highlights and next-project link.
Contact footer	"Have a project in mind?" with a pre-filled mailto link and social links.
Reload behaviour	Scroll restoration disabled so every reload starts at the top of the home page.
Responsive design	Mobile-first Tailwind classes with sm / md / lg breakpoints on every component.
404 page	Catch-all route with a link back home.

2. Technology Stack and Libraries

Every library was chosen for a specific job. Versions are read directly from `package.json`.

Library	Version	Purpose in this project
react / react-dom	19.3.1	UI library. Components, state (useState) and effects (useEffect).
vite	8.3.5	Dev server with instant hot reload and the production bundler.
@vitejs/plugin-react	6.1.2	Vite plugin that adds JSX and React Fast Refresh support.
tailwindcss	4.3.3	Utility-first CSS framework (v4). All styling and responsive breakpoints.
@tailwindcss/vite	4.3.3	Integrates Tailwind v4 directly into the Vite build.
react-router-dom	7.18.5	Client-side routing: home page, project detail pages and 404.
gsap	3.15.0	Animation engine: timelines, ScrollTrigger scroll animations, quickTo cursor follow.
@gsap/react	2.1.2	useGSAP hook: scoped animations that clean up automatically on unmount.
lenis	1.3.27	Smooth, inertia-based scrolling, used like the reference site.
react-icons	5.7.0	Brand icons (Simple Icons "Si*") for the tech stack and Feather icons for arrows.
oxlint	1.89.0	Fast linter included by the Vite template; used to keep the code warning-free.

Fonts (Anton and Roboto Flex) are loaded from Google Fonts in `index.html`. No UI component library is used: every component is hand-built with Tailwind classes, which keeps the bundle small and the design fully custom.

3. Project Setup and Development Workflow

Step 1: Scaffold the project with Vite

The project was created with the official Vite starter using the React template, as requested:

```
cd ~
npm create vite@latest abdul-rauf-portfolio -- --template react
cd abdul-rauf-portfolio
git init
```

Step 2: Install dependencies

```
npm install
npm install gsap @gsap/react lenis react-router-dom react-icons
npm install -D tailwindcss @tailwindcss/vite
```

Step 3: Clean the template

The demo files that Vite generates (`App.css`, the sample images in `src/assets` and `public/icons.svg`) were deleted, and the folders `components/layout`, `components/ui`, `sections`, `pages`, `data` and `hooks` were created.

Step 4: Build features, then verify

Configuration and global styles were written first, then the content data file, the smooth-scroll hook, the layout components, the reusable UI components, the page sections and finally the pages and routes. After each round of changes the project was built and linted, and checked in a real browser at desktop (1440px) and phone (390px) widths.

Available npm scripts

Command	What it does
`npm run dev`	Starts the Vite dev server at http://localhost:5173 with hot reload.
`npm run build`	Creates an optimised production build in the dist/ folder.
`npm run preview`	Serves the production build locally to test it.
`npm run lint`	Runs oxlint over the project.

4. Folder and File Structure

The project follows the standard Vite React layout and splits `src/` by responsibility. Content lives in one data file, so changing text never requires touching components.

```
abdul-rauf-portfolio/
├── index.html                  HTML shell, meta tags, Google Fonts
├── package.json                Dependencies and npm scripts
├── vite.config.js              Vite + React + Tailwind plugins
├── public/
│   └── favicon.svg             "AR" browser tab icon
└── src/
    ├── main.jsx                Entry point, router, reload behaviour
    ├── App.jsx                 App shell, global layout, routes
    ├── index.css               Tailwind theme, colours, fonts, globals
    ├── data/
    │   └── portfolio.js        ALL site content (single source of truth)
    ├── hooks/
    │   └── useSmoothScroll.js  Lenis + GSAP setup and scroll helpers
    ├── components/
    │   ├── layout/             Pieces visible on every page
    │   │   ├── Preloader.jsx
    │   │   ├── ParticleBackground.jsx
    │   │   ├── ScrollProgress.jsx
    │   │   ├── Navbar.jsx
    │   │   ├── StickyEmail.jsx
    │   │   └── Footer.jsx
    │   └── ui/                 Small reusable building blocks
    │       ├── Button.jsx
    │       ├── SectionTitle.jsx
    │       └── ProjectThumbnail.jsx
    ├── sections/               Home page sections, top to bottom
    │   ├── Hero.jsx
    │   ├── AboutMe.jsx
    │   ├── Skills.jsx
    │   └── Projects.jsx
    └── pages/                  Route-level screens
        ├── Home.jsx
        ├── ProjectDetails.jsx
        └── NotFound.jsx
```

Why this structure?

Folder	Responsibility
`data/`	Plain JavaScript objects holding every piece of text, link, skill and project. Components only read from here.
`hooks/`	Reusable logic that is not UI. Here: the smooth scrolling engine and helpers other components call.
`components/layout/`	Global chrome rendered once in App.jsx and present on every route.
`components/ui/`	Small, generic, reusable components with no knowledge of a specific section.
`sections/`	The large blocks that make up the home page, in the order they appear.

Folder	Responsibility
`pages/`	Top-level screens matched by React Router. A page composes sections and components.

5. Architecture and How It Works

Component tree

```
main.jsx
└── <StrictMode>
    └── <BrowserRouter>
        └── <App>                          useSmoothScroll(), ready state
            ├── <Preloader>                removed after its animation
            ├── <ParticleBackground>
            ├── <ScrollProgress>
            ├── <Navbar>
            ├── <StickyEmail>
            ├── <main> <Routes>
            │   ├── "/"                 <Home ready>
            │   │                       ├── <Hero ready>
            │   │                       ├── <AboutMe>
            │   │                       ├── <Skills>
            │   │                       └── <Projects>
            │   ├── "/projects/:slug" <ProjectDetails>
            │   └── "*"                 <NotFound>
            └── <Footer>
```

Data flow

All content is exported from `src/data/portfolio.js` as constants (`GENERAL_INFO`, `HERO`, `ABOUT`, `MY_STACK`, `STATS`, `PROJECTS`, `MENU_LINKS`, `SOCIAL_LINKS`). Components import what they need and map over arrays to render lists. Adding a new project or skill is therefore a one-line data change: the list, the hover preview, the detail page and the "next project" link all update automatically.

Load sequence and the "ready" flag

- **main.jsx** disables the browser's scroll restoration, strips any #hash from the URL and scrolls to the top, so every reload starts at the hero.
- **App** mounts. `useSmoothScroll()` creates the Lenis instance and hooks it into the GSAP ticker.
- **Preloader** plays its timeline. Just before it slides away it calls `onReveal`, which sets `ready = true` in App.
- **Hero** receives `ready`; its useGSAP effect depends on it, so the hero entrance animation starts exactly as the curtain lifts.
- When the preloader finishes, `onComplete` unmounts it completely.

Smooth scrolling and scroll animations

Lenis replaces native scrolling with a smooth, eased scroll. GSAP ScrollTrigger needs to know about every scroll update, so the hook registers `lenis.on('scroll', ScrollTrigger.update)` and drives Lenis from GSAP's own ticker (`gsap.ticker.add`). This keeps both libraries perfectly in sync. The hook also exports helpers: `scrollToSection` (used by the menu), `scrollToTop` (used on route change) and `setScrollLocked` (stops scrolling

while the menu is open).

Routing

React Router's `BrowserRouter` gives clean URLs. The menu works from any page: on the home page it smooth-scrolls to a section; on a project page it navigates to `/#section-id`, and Home scrolls to that section once it is ready, then removes the hash from the address bar so a later reload does not jump there again. On every route change App scrolls to the top and refreshes ScrollTrigger so trigger positions are recalculated for the new page.

Animation lifecycle with useGSAP

Each animated component calls `useGSAP(fn, { scope: containerRef })`. The scope means class selectors such as `'.about-word'` only match elements inside that component, and every tween and ScrollTrigger created inside is automatically reverted when the component unmounts. This prevents memory leaks and stale triggers when moving between pages.

6. Design System and Responsive Strategy

Design tokens

Tailwind CSS v4 is configured in CSS rather than a JavaScript config file. The `@theme` block in `index.css` defines custom tokens that immediately become utility classes (for example `--color-primary` becomes `bg-primary`, `text-primary`, and so on).

Token	Value	Used for
`background`	#212121	Page background
`background-light`	#303030	Menu panel, cards, subtle separators
`foreground`	#DEDEDE	Main text
`muted-foreground`	#B5B5B5	Secondary text, labels
`primary`	#00F05A	Accent: headline, buttons, hover states, progress bar
`secondary`	#00BFF0	Second gradient colour
`border`	#3D3D3D	Dividers and outlines
`font-anton`	Anton	Display headings, numbers, buttons
`font-roboto-flex`	Roboto Flex	Body text

A custom `app-container` utility centres content with a 1200px max width and responsive side padding. It was given its own name so it does not clash with Tailwind's built-in `container` class.

Responsive approach

Styles are written **mobile first**: un-prefixed classes target phones, and the `sm:` (640px), `md:` (768px) and `lg:` (1024px) prefixes progressively enhance the layout for larger screens.

Component	Phone	Tablet / Desktop
Hero	Title 6xl, stats in a 3-column row under the text	Title up to 8xl, stats in a right-hand column (lg)
Navbar panel	Full-screen width, links stacked	80vw / max 900px, Menu and Social side by side (md)
Sticky email	Hidden	Shown vertically on the left (lg)
About	Single column	12-column grid: greeting left, paragraphs right (md)
My Stack	Category name above its icons	Category in 5 columns, icons in 7 (sm)
Projects	Cover image shown inline above each project	Cursor-following preview card instead (md)

Component	Phone	Tablet / Desktop
Project details	16:9 cover, stacked overview	21:9 cover, 4 / 8 column overview (md)

Other responsive details: `min-h-svh` / `h-dvh` viewport units avoid jumps from mobile browser toolbars, long email addresses use `break-all` so they never overflow, the particle canvas resizes with the window and caps the pixel ratio at 2 for performance, and the footer and menu typography scale with each breakpoint.

Accessibility

- The menu button has a dynamic `aria-label` ("Open menu" / "Close menu") and `aria-expanded`.
- The closed menu panel uses the `inert` attribute so its links cannot be focused or read by screen readers until it opens; Escape closes it.
- Decorative elements (preloader, canvas, progress bar) are marked `aria-hidden`.
- The starfield stops animating when the user has `prefers-reduced-motion` enabled.
- Semantic headings (h1 to h5), lists and real links/buttons are used throughout.

7. Screenshots

Screenshots captured from the running site during development and testing.

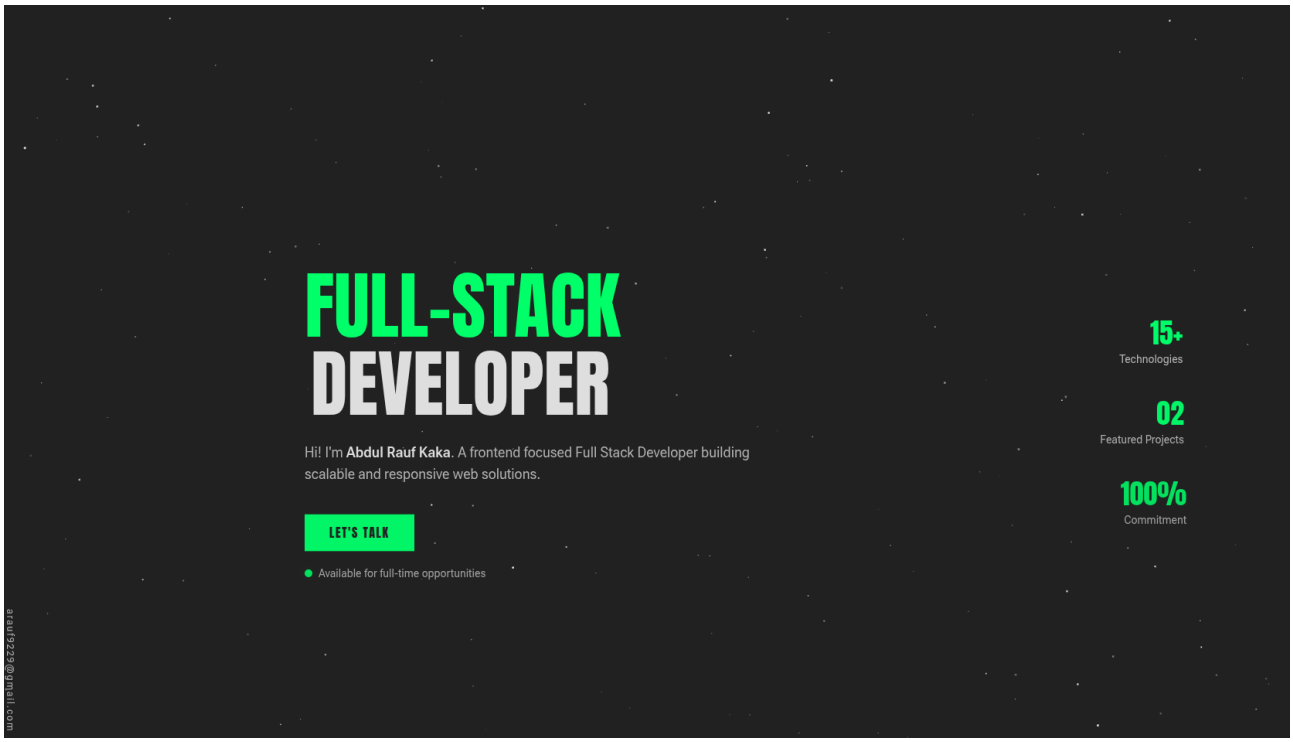


Figure 1. Desktop hero section with starfield, green headline, intro, call to action and stats.

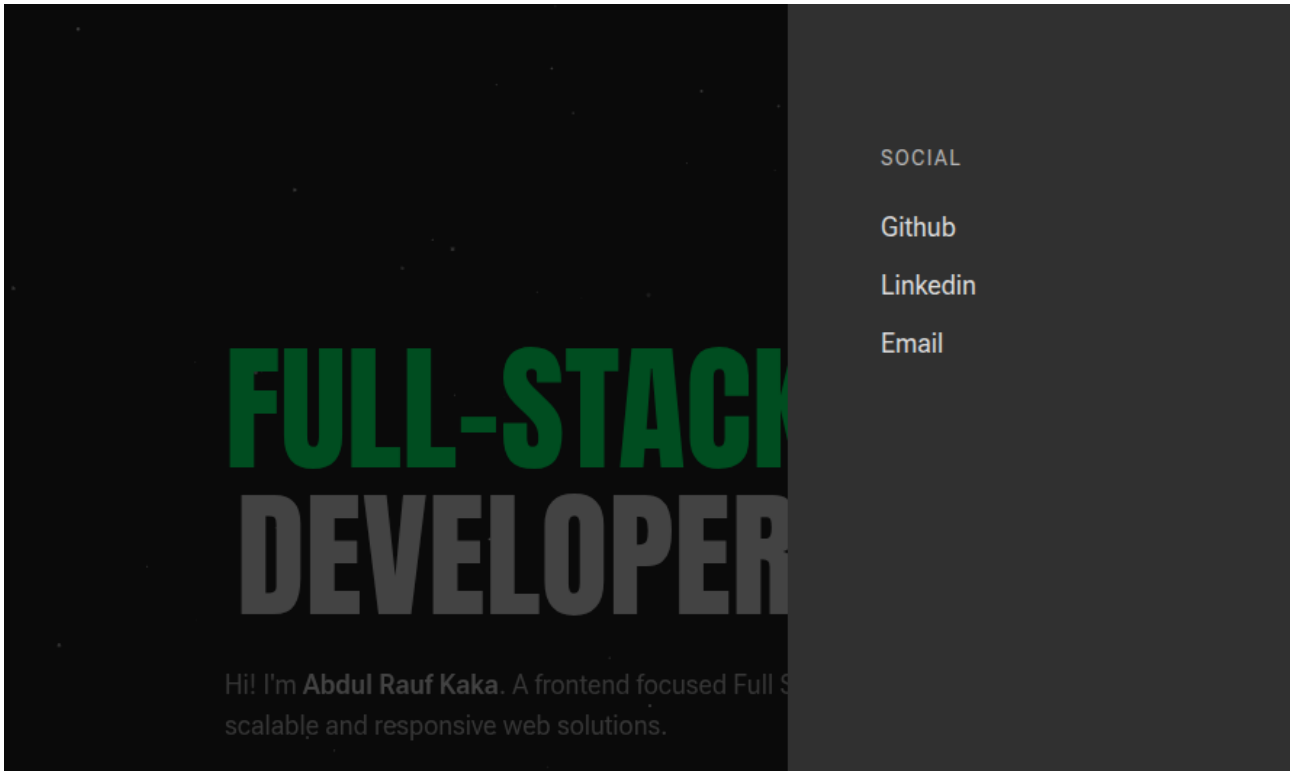


Figure 2. Slide-in navigation panel open on desktop, with Social, Menu and Get in touch columns.

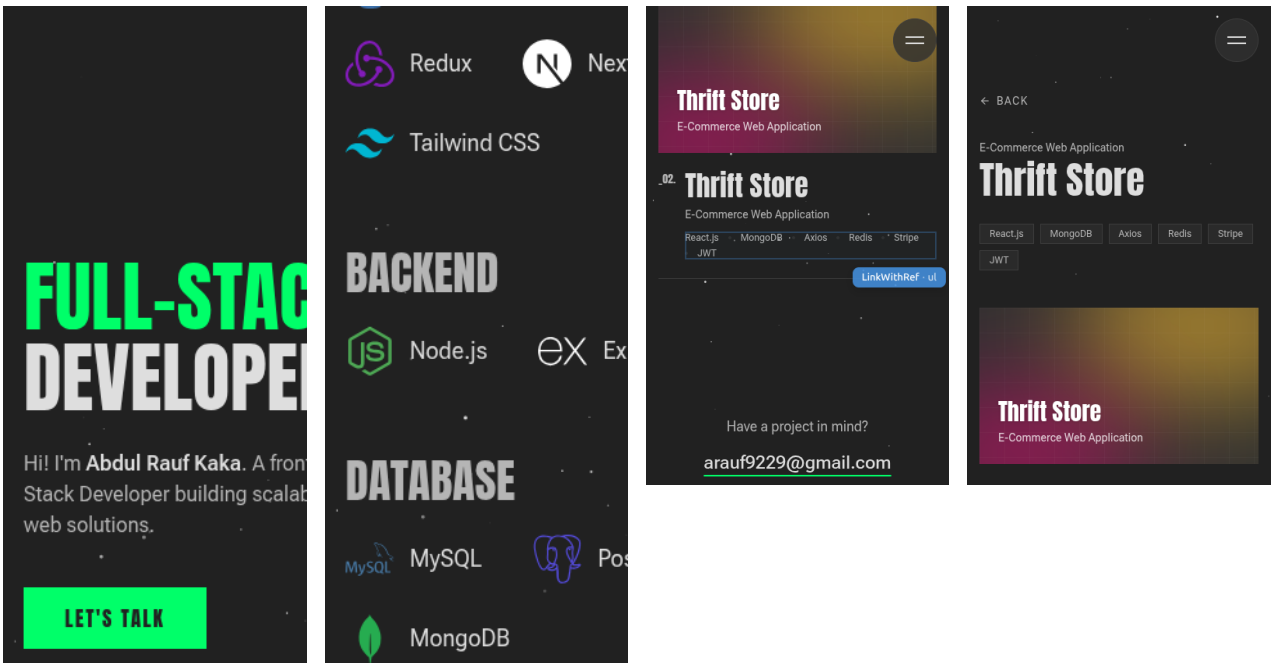


Figure 3. Phone layout (390px): hero, My Stack, a project in the list, and a project detail page.

8. Source Code Walkthrough

Every source file in the project is listed below in full, grouped by role, each preceded by an explanation of what it does and how it works. Long lines are wrapped onto an indented continuation line that keeps the same line number.

Configuration and Entry Point

`package.json`

Generated by Vite and extended by the npm install commands. **dependencies** ship to the browser; **devDependencies** (Vite, Tailwind, oxlint) are only needed to build. `"type": "module"` enables ES module syntax in config files.

```
 1 | {
 2 |   "name": "abdul-rauf-portfolio",
 3 |   "private": true,
 4 |   "version": "0.0.0",
 5 |   "type": "module",
 6 |   "scripts": {
 7 |     "dev": "vite",
 8 |     "build": "vite build",
 9 |     "lint": "oxlint",
10 |     "preview": "vite preview"
11 |   },
12 |   "dependencies": {
13 |     "@gsap/react": "^2.1.2",
14 |     "gsap": "^3.15.0",
15 |     "lenis": "^1.3.27",
16 |     "react": "^19.3.1",
17 |     "react-dom": "^19.3.1",
18 |     "react-icons": "^5.7.0",
19 |     "react-router-dom": "^7.18.5"
20 |   },
21 |   "devDependencies": {
22 |     "@tailwindcss/vite": "^4.3.3",
23 |     "@types/react": "^19.3.0",
24 |     "@types/react-dom": "^19.3.0",
25 |     "@vitejs/plugin-react": "^6.1.2",
26 |     "oxlint": "^1.89.0",
27 |     "tailwindcss": "^4.3.3",
28 |     "vite": "^8.3.5"
29 |   }
30 | }
```

`vite.config.js`

Registers two plugins: `react()` for JSX and Fast Refresh, and `tailwindcss()`, which in Tailwind v4 replaces the old PostCSS setup and scans source files for class names automatically.

```
1 | import tailwindcss from '@tailwindcss/vite'
2 | import react from '@vitejs/plugin-react'
3 | import { defineConfig } from 'vite'
4 |
5 | // https://vite.dev/config/
6 | export default defineConfig({
7 |   plugins: [react(), tailwindcss()],
8 | })
```

`index.html`

The single HTML page Vite serves. It sets the title, description and theme colour, preconnects to Google Fonts and loads **Anton** and the variable **Roboto Flex** font. React renders into `<div id="root">`, and `/src/main.jsx` is loaded as a module.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="theme-color" content="#212121" />
    <meta
      name="description"
      content="Abdul Rauf Kaka - Frontend focused Full Stack Developer building scalable and
        responsive web solutions."
    />
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link
      href="https://fonts.googleapis.com/css2?family=Anton&family=Roboto+Flex:opsz,wght@8..144,10
        0..1000&display=swap"
      rel="stylesheet"
    />
    <title>Portfolio - Abdul Rauf Kaka</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

`public/favicon.svg`

A small SVG "AR" monogram in the site colours used as the browser tab icon.

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">
  <rect width="64" height="64" rx="12" fill="#212121"/>
  <text x="32" y="44" text-anchor="middle" font-family="Anton, Impact, sans-serif" font-size="34"
    fill="#00F05A">AR</text>
</svg>
```

`src/main.jsx`

The JavaScript entry point. Before React starts it sets `history.scrollRestoration = 'manual'` so the browser does not restore the old scroll position on reload, removes any URL hash with `replaceState`, and scrolls to the top. This guarantees a reload always lands on the start of the home page.

It then renders `<App />` inside `StrictMode` (extra development checks) and `BrowserRouter` (enables routing everywhere).

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App.jsx'
import './index.css'

if ('scrollRestoration' in window.history) {
  window.history.scrollRestoration = 'manual'
}
if (window.location.hash) {
  window.history.replaceState(null, '', window.location.pathname + window.location.search)
```

```
}
window.scrollTo(0, 0)

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

src/index.css

`@import 'tailwindcss'` loads Tailwind v4. The `@theme` block defines the colour palette, the two font families and a blink animation as design tokens.

The `@layer base` block applies the background, text colour and font to the body, hides horizontal overflow, colours text selection green and styles a thin scrollbar.

The `@utility app-container` rule creates the centred, padded content wrapper used by every section. The final rules are the small CSS snippet Lenis requires to work correctly.

```
@import 'tailwindcss';

@theme {
  --color-background: #212121;
  --color-background-light: #303030;
  --color-foreground: #dedede;
  --color-muted-foreground: #b5b5b5;
  --color-primary: #00f05a;
  --color-secondary: #00bff0;
  --color-border: #3d3d3d;

  --font-anton: 'Anton', 'Impact', sans-serif;
  --font-roboto-flex: 'Roboto Flex', system-ui, sans-serif;

  --animate-blink: blink 1.2s ease-in-out infinite;

  @keyframes blink {
    0%,
    100% {
      opacity: 1;
    }
    50% {
      opacity: 0.25;
    }
  }
}

@layer base {
  html {
    scroll-behavior: auto;
  }

  body {
    @apply bg-background text-foreground font-roboto-flex antialiased;
    overflow-x: hidden;
  }

  ::selection {
    @apply bg-primary text-background;
  }

  ::-webkit-scrollbar {
    width: 6px;
  }
  ::-webkit-scrollbar-track {
    @apply bg-background;
```

```
  }
  ::-webkit-scrollbar-thumb {
    @apply bg-background-light rounded-full;
  }
}

@utility app-container {
  margin-inline: auto;
  padding-inline: 1rem;
  width: 100%;
  max-width: 1200px;

  @variant md {
    padding-inline: 2rem;
  }
}

/* Lenis smooth scroll */
html.lenis,
html.lenis body {
  height: auto;
}
.lenis.lenis-smooth {
  scroll-behavior: auto !important;
}
.lenis.lenis-stopped {
  overflow: hidden;
}
```

src/App.jsx

The application shell. It holds two pieces of state: `ready` (set when the preloader starts revealing the page) and `showPreloader` (set to false to unmount it). It calls `useSmoothScroll()` once for the whole app.

An effect watches `pathname` and `hash`: on every route change it jumps to the top (unless a section hash was requested) and calls `ScrollTrigger.refresh()` after layout settles.

It renders the global layout pieces, then <Routes> with three routes: `/`, `/projects/:slug` and a * catch-all for the 404 page, followed by the footer.

```
import { ScrollTrigger } from 'gsap/ScrollTrigger'
import { useEffect, useState } from 'react'
import { Route, Routes, useLocation } from 'react-router-dom'
import Footer from './components/layout/Footer'
import Navbar from './components/layout/Navbar'
import ParticleBackground from './components/layout/ParticleBackground'
import Preloader from './components/layout/Preloader'
import ScrollProgress from './components/layout/ScrollProgress'
import StickyEmail from './components/layout/StickyEmail'
import useSmoothScroll, { scrollToTop } from './hooks/useSmoothScroll'
import Home from './pages/Home'
import NotFound from './pages/NotFound'
import ProjectDetails from './pages/ProjectDetails'

export default function App() {
  const [ready, setReady] = useState(false)
  const [showPreloader, setShowPreloader] = useState(true)
  const { pathname, hash } = useLocation()

  useSmoothScroll()

  useEffect(() => {
    if (!hash) scrollToTop(true)
    const timer = setTimeout(() => ScrollTrigger.refresh(), 300)
    return () => clearTimeout(timer)
  }, [pathname, hash])

```

```
  return (
    <>
      {showPreloader && (
        <Preloader onReveal={() => setReady(true)} onComplete={() => setShowPreloader(false)} />
      )}
      <ParticleBackground />
      <ScrollProgress />
      <Navbar />
      <StickyEmail />

      <main>
        <Routes>
          <Route path="/" element={<Home ready={ready} />} />
          <Route path="/projects/:slug" element={<ProjectDetails />} />
          <Route path="*" element={<NotFound />} />
        </Routes>
      </main>

      <Footer />
    </>
  )
}
```

Content Data

`src/data/portfolio.js`

The single source of truth for all text and content. It imports brand icons from `react-icons/si` and pairs each with its official brand colour.

GENERAL_INFO: name and email plus a default subject and body for the contact mailto link. **SOCIAL_LINKS**: GitHub, LinkedIn (placeholders to fill in) and email. **MENU_LINKS**: the menu labels and the section id each one scrolls to.

HERO and **ABOUT** hold the bio text supplied by Abdul Rauf. **MY_STACK** groups skills into frontend, backend, database and tools. **STATS** computes the technology count from MY_STACK automatically.

PROJECTS holds SatIntel (the SUPARCO final year project) and the Thrift Store e-commerce app. Each project has a `slug` used in its URL, a subtitle, year, tech stack, a gradient used for its generated cover, optional `liveUrl` / `sourceUrl` / `image`, a description and highlights.

```
import {
  SiDocker,
  SiExpress,
  SiGit,
  SiGithub,
  SiJavascript,
  SiMongodb,
  SiMysql,
  SiNextdotjs,
  SiNodedotjs,
  SiPostgresql,
  SiPython,
  SiReact,
  SiRedux,
  SiTailwindcss,
  SiTypescript,
} from 'react-icons/si'

export const GENERAL_INFO = {
  name: 'Abdul Rauf Kaka',
  firstName: 'Abdul Rauf',
  email: 'arauf9229@gmail.com',
```

```
  emailSubject: "Let's collaborate on a project",
  emailBody: 'Hi Abdul Rauf, I came across your portfolio and would like to talk about...',
}

// Replace the `#` links with your real profile URLs.
export const SOCIAL_LINKS = [
  { name: 'github', url: '#' },
  { name: 'linkedin', url: '#' },
  { name: 'email', url: `mailto:${GENERAL_INFO.email}` },
]

export const MENU_LINKS = [
  { name: 'Home', id: 'banner' },
  { name: 'About Me', id: 'about-me' },
  { name: 'My Stack', id: 'my-stack' },
  { name: 'Projects', id: 'selected-projects' },
]

export const HERO = {
  titleTop: 'FULL-STACK',
  titleBottom: 'DEVELOPER',
  intro:
    'A frontend focused Full Stack Developer building scalable and responsive web solutions.',
  availability: 'Available for full-time opportunities',
}

export const ABOUT = {
  statement:
    'I believe in a user centered design approach, ensuring that every project I work on is
      tailored to meet the specific needs of its users.',
  greeting: "Hi, I'm Abdul Rauf Kaka.",
  paragraphs: [
    "I'm a frontend-focused full-stack developer dedicated to turning ideas into creative
      solutions. I specialize in creating seamless and intuitive user experiences.",
    'My approach focuses on creating scalable, high-performing solutions tailored to both user
      needs and business objectives. By prioritizing performance, accessibility, and
      responsiveness, I strive to deliver experiences that not only engage users but also drive
      tangible results.',
  ],
}

export const MY_STACK = {
  frontend: [
    { name: 'TypeScript', icon: SiTypescript, color: '#3178C6' },
    { name: 'JavaScript', icon: SiJavascript, color: '#F7DF1E' },
    { name: 'Python', icon: SiPython, color: '#3776AB' },
    { name: 'React.js', icon: SiReact, color: '#61DAFB' },
    { name: 'Redux', icon: SiRedux, color: '#764ABC' },
    { name: 'Next.js', icon: SiNextdotjs, color: '#FFFFFF' },
    { name: 'Tailwind CSS', icon: SiTailwindcss, color: '#06B6D4' },
  ],
  backend: [
    { name: 'Node.js', icon: SiNodedotjs, color: '#5FA04E' },
    { name: 'Express.js', icon: SiExpress, color: '#FFFFFF' },
  ],
  database: [
    { name: 'MySQL', icon: SiMysql, color: '#4479A1' },
    { name: 'PostgreSQL', icon: SiPostgresql, color: '#4169E1' },
    { name: 'MongoDB', icon: SiMongodb, color: '#47A248' },
  ],
  tools: [
    { name: 'Git', icon: SiGit, color: '#F05032' },
    { name: 'GitHub', icon: SiGithub, color: '#FFFFFF' },
    { name: 'Docker', icon: SiDocker, color: '#2496ED' },
  ],
}

export const STATS = [
  { value: `${Object.values(MY_STACK).flat().length}+`, label: 'Technologies' },
  { value: '02', label: 'Featured Projects' },
  { value: '100%', label: 'Commitment' },
]
```

```
export const PROJECTS = [
  {
    slug: 'satintel',
    title: 'SatIntel',
    subtitle: 'Final Year Project with SUPARCO Resolve Lab',
    year: '2025',
    techStack: ['Multimodal AI', 'RemoteCLIP', 'Phi-3.5-Mini', 'LoRA', 'GroundingDINO', 'SAM2'],
    gradient: ['#00F05A', '#00BFF0'],
    // Optional: set to a URL to show "Live Preview" / "Source Code" buttons.
    liveUrl: '',
    sourceUrl: '',
    description:
      'An end-to-end multimodal AI for satellite image intelligence, developed in collaboration with SUPARCO.',
    highlights: [
      'Integrated RemoteCLIP as a vision encoder with Phi-3.5-Mini (3.8B LLM) via a custom MLP projector.',
      'Trained across a 3-stage pipeline: Feature Alignment and Visual Instruction Tuning.',
      'Applied LoRA fine-tuning for efficient GPU training.',
      'Incorporated GroundingDINO and SAM2 for zero-shot object detection and segmentation via autonomous tool routing.',
      'Evaluated on RSICD and RSVQA-LR benchmarks, achieving results comparable to GeoChat and LLaVA-RE.',
    ],
  },
  {
    slug: 'thrift-store',
    title: 'Thrift Store',
    subtitle: 'E-Commerce Web Application',
    year: '2024',
    techStack: ['React.js', 'MongoDB', 'Axios', 'Redis', 'Stripe', 'JWT'],
    gradient: ['#F7B500', '#FF3D7F'],
    liveUrl: '',
    sourceUrl: '',
    description:
      'A full-stack e-commerce application with dynamic product listings, secure authentication and real-time payments.',
    highlights: [
      'Developed a full-stack e-commerce application using React.js, implementing dynamic product listings, client-side routing, and efficient state management.',
      'Implemented secure user authentication and authorization using JWT access and refresh tokens to improve session management and data protection.',
      'Integrated Stripe payment gateway to enable secure payment and checkout processing with real-time transaction handling.',
      'Leveraged Redis caching to optimize data retrieval for frequently accessed endpoints, reducing server response latency.',
      'Used Axios and React Hooks for efficient API communication and asynchronous state management across application workflows.',
    ],
  },
]
```

Hooks

`src/hooks/useSmoothScroll.js`

Registers the GSAP ScrollTrigger plugin and keeps a module-level reference to the active Lenis instance so other components can control scrolling without prop drilling.

The default-exported hook creates Lenis with `lerp: 0.1` (how quickly the scroll eases toward its target), forwards Lenis scroll events to ScrollTrigger, and runs `lenis.raf` from the GSAP ticker. `lagSmoothing(0)` prevents GSAP from skipping frames after a lag spike so scroll and animation never drift apart. The cleanup function removes the ticker callback and destroys Lenis.

Helper exports: `scrollToSection(id)` smooth-scrolls to an element (falling back to native scrolling), `scrollToTop(immediate)` and `setScrollLocked(bool)`.

```
import gsap from 'gsap'
import { ScrollTrigger } from 'gsap/ScrollTrigger'
import Lenis from 'lenis'
import { useEffect } from 'react'

gsap.registerPlugin(ScrollTrigger)

let lenisInstance = null

export const scrollToSection = (target, options = {}) => {
  const el = typeof target === 'string' ? document.getElementById(target) : target
  if (!el) return
  if (lenisInstance) {
    lenisInstance.scrollTo(el, { offset: 0, duration: 1.4, ...options })
  } else {
    el.scrollIntoView({ behavior: 'smooth' })
  }
}

export const scrollToTop = (immediate = false) => {
  if (lenisInstance) lenisInstance.scrollTo(0, { immediate })
  else window.scrollTo({ top: 0, behavior: immediate ? 'auto' : 'smooth' })
}

export const setScrollLocked = (locked) => {
  if (!lenisInstance) return
  if (locked) lenisInstance.stop()
  else lenisInstance.start()
}

export default function useSmoothScroll() {
  useEffect(() => {
    const lenis = new Lenis({ lerp: 0.1, smoothWheel: true })
    lenisInstance = lenis

    lenis.on('scroll', ScrollTrigger.update)
    const tick = (time) => lenis.raf(time * 1000)
    gsap.ticker.add(tick)
    gsap.ticker.lagSmoothing(0)

    return () => {
      gsap.ticker.remove(tick)
      lenis.destroy()
      lenisInstance = null
    }
  }, [])
}
```

Layout Components

`src/components/layout/Preloader.jsx`

A full-screen black overlay with the letters "ABDUL RAUF", each in its own span so it can be animated individually. The GSAP timeline: letters rise into view with a 0.06s stagger, pause, rise out of view, then `.add(onReveal)` tells App to start the hero animation, and finally the whole overlay slides up (`yPercent: -100`). When the timeline completes, `onComplete` removes the component. Spaces are rendered as non-breaking spaces so they keep their width.

```
import { useGSAP } from '@gsap/react'
import gsap from 'gsap'
import { useRef } from 'react'
```

```
 4 |
 5 | const NAME = 'ABDUL RAUF'
 6 |
 7 | export default function Preloader({ onReveal, onComplete }) {
 8 |   const containerRef = useRef(null)
 9 |
10 |   useGSAP(
11 |     () => {
12 |       const tl = gsap.timeline({ defaults: { ease: 'power3.out' }, onComplete })
13 |
14 |       tl.from('.preloader-letter', {
15 |         yPercent: 120,
16 |         opacity: 0,
17 |         stagger: 0.06,
18 |         duration: 0.7,
19 |       })
20 |         .to('.preloader-letter', {
21 |           yPercent: -120,
22 |           opacity: 0,
23 |           stagger: 0.04,
24 |           duration: 0.5,
25 |           ease: 'power3.in',
26 |           delay: 0.4,
27 |         })
28 |         .add(() => onReveal?.())
29 |         .to(containerRef.current, {
30 |           yPercent: -100,
31 |           duration: 0.8,
32 |           ease: 'power4.inOut',
33 |         })
34 |     },
35 |     { scope: containerRef },
36 |   )
37 |
38 |   return (
39 |     <div
40 |       ref={containerRef}
41 |       className="fixed inset-0 z-[100] flex items-center justify-center bg-black"
42 |       aria-hidden="true"
43 |     >
44 |       <h2 className="flex overflow-hidden font-anton text-5xl text-foreground sm:text-7xl
   |         md:text-8xl">
45 |         {NAME.split('').map((letter, i) => (
46 |           <span key={i} className="preloader-letter inline-block">
47 |             {letter === ' ' ? '\u00A0' : letter}
48 |           </span>
49 |         ))}
50 |       </h2>
51 |     </div>
52 |   )
53 | }
```

`src/components/layout/ParticleBackground.jsx`

Draws the starfield on a fixed, full-screen <canvas> placed behind all content (`-z-10`) and ignoring pointer events.

The number of particles scales with screen area (one per 9,000 px²). Each particle has a random position, radius, upward speed and opacity. Every animation frame it moves up, its opacity twinkles between bounds, and when it leaves the top it re-enters from the bottom at a random x position.

The canvas is sized for the device pixel ratio (capped at 2) so stars are crisp on retina screens without wasting performance. It rebuilds on resize, and only draws one static frame when the user prefers reduced motion.

```
1 | import { useEffect, useRef } from 'react'
```

```
 2 |
 3 | export default function ParticleBackground() {
 4 |   const canvasRef = useRef(null)
 5 |
 6 |   useEffect(() => {
 7 |     const canvas = canvasRef.current
 8 |     const ctx = canvas.getContext('2d')
 9 |     const reduceMotion = window.matchMedia('(prefers-reduced-motion: reduce)').matches
10 |     let particles = []
11 |     let frameId
12 |
13 |     const createParticles = () => {
14 |       const dpr = Math.min(window.devicePixelRatio || 1, 2)
15 |       canvas.width = window.innerWidth * dpr
16 |       canvas.height = window.innerHeight * dpr
17 |       canvas.style.width = `${window.innerWidth}px`
18 |       canvas.style.height = `${window.innerHeight}px`
19 |       ctx.setTransform(dpr, 0, 0, dpr, 0, 0)
20 |
21 |       const count = Math.round((window.innerWidth * window.innerHeight) / 9000)
22 |       particles = Array.from({ length: count }, () => ({
23 |         x: Math.random() * window.innerWidth,
24 |         y: Math.random() * window.innerHeight,
25 |         r: Math.random() * 1.2 + 0.2,
26 |         speed: Math.random() * 0.25 + 0.05,
27 |         alpha: Math.random() * 0.6 + 0.2,
28 |         twinkle: Math.random() * 0.02 + 0.005,
29 |       }))
30 |     }
31 |
32 |     const draw = () => {
33 |       ctx.clearRect(0, 0, window.innerWidth, window.innerHeight)
34 |       for (const p of particles) {
35 |         p.y -= p.speed
36 |         p.alpha += p.twinkle
37 |         if (p.alpha > 0.9 || p.alpha < 0.15) p.twinkle *= -1
38 |         if (p.y < -5) {
39 |           p.y = window.innerHeight + 5
40 |           p.x = Math.random() * window.innerWidth
41 |         }
42 |         ctx.beginPath()
43 |         ctx.arc(p.x, p.y, p.r, 0, Math.PI * 2)
44 |         ctx.fillStyle = `rgba(255, 255, 255, ${p.alpha})`
45 |         ctx.fill()
46 |       }
47 |       if (!reduceMotion) frameId = requestAnimationFrame(draw)
48 |     }
49 |
50 |     createParticles()
51 |     draw()
52 |     window.addEventListener('resize', createParticles)
53 |
54 |     return () => {
55 |       cancelAnimationFrame(frameId)
56 |       window.removeEventListener('resize', createParticles)
57 |     }
58 |   }, [])
59 |
60 |   return (
61 |     <canvas
62 |       ref={canvasRef}
63 |       className="pointer-events-none fixed inset-0 -z-10"
64 |       aria-hidden="true"
65 |     />
66 |   )
67 | }
```

src/components/layout/ScrollProgress.jsx

A 3px green-to-cyan bar fixed to the top of the viewport. A passive scroll listener computes `scrollY / (scrollHeight - innerHeight)` and writes it straight to the element's `transform: scaleX()` through a ref. Updating the DOM directly instead of React state avoids re-rendering on every scroll event.

```
import { useEffect, useRef } from 'react'

export default function ScrollProgress() {
  const barRef = useRef(null)

  useEffect(() => {
    const update = () => {
      const max = document.documentElement.scrollHeight - window.innerHeight
      const progress = max > 0 ? window.scrollY / max : 0
      if (barRef.current) barRef.current.style.transform = `scaleX(${progress})`
    }
    update()
    window.addEventListener('scroll', update, { passive: true })
    window.addEventListener('resize', update)
    return () => {
      window.removeEventListener('scroll', update)
      window.removeEventListener('resize', update)
    }
  }, [])

  return (
    <div className="fixed inset-x-0 top-0 z-50 h-[3px]" aria-hidden="true">
      <div
        ref={barRef}
        className="h-full origin-left scale-x-0 bg-gradient-to-r from-primary to-secondary"
      />
    </div>
  )
}
```

src/components/layout/Navbar.jsx

The round hamburger button is fixed in the top-right corner above everything (`z-[60]`). Its two bars are absolutely positioned spans that rotate into an X when `isOpen` is true, using CSS transitions.

A semi-transparent backdrop and the slide-in <nav> panel are toggled with `translate-x-full` / `translate-x-0` and a custom cubic-bezier easing. While open, Lenis scrolling is stopped and an Escape key listener closes the menu.

`handleNavClick` closes the menu, then either smooth-scrolls to the section (after a short delay so the panel animation finishes) or, from another page, navigates to `/#id`. Each menu item has a coloured dot that grows on hover.

```
import { useEffect, useState } from 'react'
import { useLocation, useNavigate } from 'react-router-dom'
import { GENERAL_INFO, MENU_LINKS, SOCIAL_LINKS } from '../../data/portfolio'
import { scrollToSection, setScrollLocked } from '../../hooks/useSmoothScroll'

const MENU_COLORS = ['#00F05A', '#00BFF0', '#F7B500', '#FF3D7F']

export default function Navbar() {
  const [isOpen, setIsOpen] = useState(false)
  const navigate = useNavigate()
  const { pathname } = useLocation()

  useEffect(() => {
    setScrollLocked(isOpen)
```

```
15 |     const onKeyDown = (e) => e.key === 'Escape' && setIsOpen(false)
16 |     window.addEventListener('keydown', onKeyDown)
17 |     return () => window.removeEventListener('keydown', onKeyDown)
18 |   }, [isOpen])
19 |
20 |   const handleNavClick = (id) => {
21 |     setIsOpen(false)
22 |     if (pathname !== '/') {
23 |       navigate(`/#${id}`)
24 |       return
25 |     }
26 |     setTimeout(() => scrollToSection(id), 300)
27 |   }
28 |
29 |   return (
30 |     <>
31 |       <button
32 |         type="button"
33 |         onClick={() => setIsOpen((open) => !open)}
34 |         aria-label={isOpen ? 'Close menu' : 'Open menu'}
35 |         aria-expanded={isOpen}
36 |         className="group fixed right-4 top-4 z-[60] flex size-14 items-center justify-center
   |           rounded-full border border-border bg-background-light/80 backdrop-blur transition-colors
   |           hover:border-primary md:right-10 md:top-10"
37 |       >
38 |         <span className="sr-only">Menu</span>
39 |         <span
40 |           className={`absolute h-0.5 w-6 bg-foreground transition-all duration-300 ${
41 |             isOpen ? 'rotate-45' : '-translate-y-[4px] group-hover:w-4'
42 |           }`}
43 |         />
44 |         <span
45 |           className={`absolute h-0.5 w-6 bg-foreground transition-all duration-300 ${
46 |             isOpen ? '-rotate-45' : 'translate-y-[4px]'
47 |           }`}
48 |         />
49 |       </button>
50 |
51 |       {/* Backdrop */}
52 |       <div
53 |         onClick={() => setIsOpen(false)}
54 |         className={`fixed inset-0 z-[55] bg-black/70 transition-opacity duration-500 ${
55 |           isOpen ? 'opacity-100' : 'pointer-events-none opacity-0'
56 |         }`}
57 |       />
58 |
59 |       {/* Slide-in panel */}
60 |       <nav
61 |         inert={!isOpen}
62 |         className={`fixed right-0 top-0 z-[58] flex h-dvh w-full flex-col overflow-y-auto
   |           bg-background-light px-8 py-24 transition-transform duration-700
   |           ease-[cubic-bezier(0.77,0,0.18,1)] sm:px-12 md:w-[min(80vw,900px)] md:px-16 ${
63 |           isOpen ? 'translate-x-0' : 'translate-x-full'
64 |         }`}
65 |       >
66 |         <div className="flex grow flex-col justify-between gap-12 md:flex-row-reverse
   |           md:items-start">
67 |           <div>
68 |             <p className="mb-6 text-sm uppercase tracking-widest text-muted-foreground">Menu</p>
69 |             <ul className="space-y-3">
70 |               {MENU_LINKS.map((link, i) => (
71 |                 <li key={link.id}>
72 |                   <button
73 |                     type="button"
74 |                     tabIndex={isOpen ? 0 : -1}
75 |                     onClick={() => handleNavClick(link.id)}
76 |                     className="group flex items-center gap-4 text-left font-anton text-4xl
   |                       uppercase transition-colors sm:text-5xl"
77 |                   >
78 |                     <span
79 |                       className="block size-3.5 rounded-full transition-transform duration-300
   |                         group-hover:scale-150"
80 |                       style={{ backgroundColor: MENU_COLORS[i % MENU_COLORS.length] }}
```

```
                    />
                    <span className="transition-transform duration-300 group-hover:translate-x-2">
                      {link.name}
                    </span>
                  </button>
                </li>
              ))}
            </ul>
          </div>

          <div>
            <p className="mb-6 text-sm uppercase tracking-widest text-muted-foreground">Social</p>
            <ul className="space-y-3">
              {SOCIAL_LINKS.map((link) => (
                <li key={link.name}>
                  <a
                    href={link.url}
                    target={link.url.startsWith('http') ? '_blank' : undefined}
                    rel="noreferrer"
                    tabIndex={isOpen ? 0 : -1}
                    className="text-lg capitalize text-foreground transition-colors
                      hover:text-primary"
                  >
                    {link.name}
                  </a>
                </li>
              ))}
            </ul>
          </div>
        </div>

        <div className="mt-16">
          <p className="mb-3 text-sm uppercase tracking-widest text-muted-foreground">Get in
            touch</p>
          <a
            href={`mailto:${GENERAL_INFO.email}`}
            tabIndex={isOpen ? 0 : -1}
            className="break-all text-lg transition-colors hover:text-primary sm:text-xl"
          >
            {GENERAL_INFO.email}
          </a>
        </div>
      </nav>
    </>
  )
}
```

src/components/layout/StickyEmail.jsx

The vertical email link fixed to the left edge, reproduced from the reference site. `writing-mode: vertical-rl` rotates the text. It is hidden below the `lg` breakpoint where there is not enough horizontal space.

```
import { GENERAL_INFO } from '../../data/portfolio'

export default function StickyEmail() {
  return (
    <div className="fixed bottom-32 left-0 z-40 hidden lg:block">
      <a
        href={`mailto:${GENERAL_INFO.email}`}
        className="block px-3 text-xs tracking-[0.15em] text-muted-foreground transition-colors
          hover:text-primary"
        style={{ writingMode: 'vertical-rl', textOrientation: 'mixed' }}
      >
        {GENERAL_INFO.email}
      </a>
    </div>
  )
}
```

src/components/layout/Footer.jsx

The contact area at the bottom of every page: "Have a project in mind?", the email as a large underlined link, and the social links. The mailto link pre-fills a subject and message using `encodeURIComponent` so the visitor's email client opens ready to send.

```
import { GENERAL_INFO, SOCIAL_LINKS } from '../../data/portfolio'

export default function Footer() {
  return (
    <footer className="relative overflow-hidden pt-24 pb-16 text-center" id="contact">
      <div className="app-container">
        <p className="text-lg text-muted-foreground">Have a project in mind?</p>
        <a
          href={`mailto:${GENERAL_INFO.email}?subject=${encodeURIComponent(
            GENERAL_INFO.emailSubject,
          )}&body=${encodeURIComponent(GENERAL_INFO.emailBody)}`}
          className="mt-4 inline-block break-all text-2xl font-medium text-foreground underline
            decoration-primary decoration-2 underline-offset-8 transition-colors hover:text-primary
            sm:text-3xl md:text-4xl"
        >
          {GENERAL_INFO.email}
        </a>

        <ul className="mt-10 flex justify-center gap-6">
          {SOCIAL_LINKS.map((link) => (
            <li key={link.name}>
              <a
                href={link.url}
                target={link.url.startsWith('http') ? '_blank' : undefined}
                rel="noreferrer"
                className="text-sm uppercase tracking-widest text-muted-foreground
                  transition-colors hover:text-primary"
              >
                {link.name}
              </a>
            </li>
          ))}
        </ul>
      </div>
    </footer>
  )
}
```

Reusable UI Components

src/components/ui/Button.jsx

One button style used across the site, rendered as an <a>, a React Router <Link> or a <button> depending on the as prop. Two variants exist: a solid green **primary** and an **outline**. Any other props (href, to, target) are passed straight through.

```
import { Link } from 'react-router-dom'

const VARIANTS = {
  primary: 'bg-primary text-background hover:bg-foreground',
  outline: 'border border-foreground/40 text-foreground hover:border-primary hover:text-primary',
}

export default function Button({ as = 'a', variant = 'primary', className = '', children,
  ...props }) {
  const classes = `group relative inline-flex h-12 items-center justify-center gap-2
    overflow-hidden px-8 font-anton text-base uppercase tracking-widest transition-colors
    duration-300 ${VARIANTS[variant]} ${className}`

  if (as === 'link') {
```

```
    return (
      <Link className={classes} {...props}>
        {children}
      </Link>
    )
  }
  if (as === 'button') {
    return (
      <button type="button" className={classes} {...props}>
        {children}
      </button>
    )
  }
  return (
    <a className={classes} {...props}>
      {children}
    </a>
  )
}
```

src/components/ui/SectionTitle.jsx

The small section label used by My Stack and Selected Projects: an asterisk-shaped SVG that slowly spins (Tailwind arbitrary animation `animate-[spin_7s_linear_infinite]`) next to an uppercase title, matching the reference site.

```
export default function SectionTitle({ title, className = '' }) {
  return (
    <div className={`mb-10 flex items-center gap-4 ${className}`}>
      <svg
        width="22"
        height="22"
        viewBox="0 0 24 24"
        className="animate-[spin_7s_linear_infinite] fill-muted-foreground"
        aria-hidden="true"
      >
        <path d="M11 0h2v9.17l6.48-6.48 1.42 1.42L14.83 10.5H24v2h-9.17l6.48 6.48-1.42 1.42L13
          13.83V24h-2v-9.17l-6.48 6.48-1.42-1.42 6.07-6.06H0v-2h9.17L3.1 4.1l1.42-1.42L11 9.17z" />
      </svg>
      <h2 className="text-xl uppercase leading-none">{title}</h2>
    </div>
  )
}
```

src/components/ui/ProjectThumbnail.jsx

Produces a project cover. If the project defines an `image`, it renders that image. Otherwise it generates artwork: two large blurred circles in the project's gradient colours, a faint grid pattern drawn with CSS gradients, and the project title and subtitle. This gives every project an attractive, on-brand cover even before real screenshots exist.

```
/**
 * Generated artwork used as a project cover. Swap it for a real
 * screenshot by giving the project an `image` field in data/portfolio.js.
 */
export default function ProjectThumbnail({ project, className = '' }) {
  const [from, to] = project.gradient

  if (project.image) {
    return (
      <img
        src={project.image}
        alt={project.title}
        className={`h-full w-full object-cover ${className}`}
        loading="lazy"
      />
```

```
    )
  }

  return (
    <div
      className={`relative flex h-full w-full items-end overflow-hidden bg-background-light p-6
        ${className}`}
      role="img"
      aria-label={`${project.title} cover`}
    >
      <div
        className="absolute -right-1/4 -top-1/3 size-[90%] rounded-full opacity-60 blur-3xl"
        style={{ background: from }}
      />
      <div
        className="absolute -bottom-1/3 -left-1/4 size-[80%] rounded-full opacity-50 blur-3xl"
        style={{ background: to }}
      />
      <div
        className="absolute inset-0 opacity-20"
        style={{
          backgroundImage:
            'linear-gradient(rgba(255,255,255,.15) 1px, transparent 1px), linear-gradient(90deg,
              rgba(255,255,255,.15) 1px, transparent 1px)',
          backgroundSize: '32px 32px',
        }}
      />
      <div className="relative">
        <p className="font-anton text-3xl leading-none text-white drop-shadow sm:text-5xl">
          {project.title}
        </p>
        <p className="mt-2 text-sm text-white/80">{project.subtitle}</p>
      </div>
    </div>
  )
}
```

Home Page Sections

src/sections/Hero.jsx

Each title line sits inside an `overflow-hidden` wrapper and is animated from `yPercent: 110`, creating a "rising from behind a mask" reveal. The intro, button and availability badge then fade up, and the stats slide in from the right.

The effect only runs once `ready` is true (it is listed in `dependencies`), so the animation is not wasted behind the preloader.

A second, scrubbed ScrollTrigger fades, shrinks and lifts the hero content as the user scrolls past it. The availability dot uses Tailwind's `animate-ping` for a pulsing effect.

```
import { useGSAP } from '@gsap/react'
import gsap from 'gsap'
import { useRef } from 'react'
import Button from '../components/ui/Button'
import { GENERAL_INFO, HERO, STATS } from '../data/portfolio'

export default function Hero({ ready }) {
  const containerRef = useRef(null)

  useGSAP(
    () => {
      if (!ready) return
      const tl = gsap.timeline({ defaults: { ease: 'power3.out', duration: 0.9 } })
      tl.from('.hero-title-line', { yPercent: 110, stagger: 0.12 })
        .from('.hero-fade', { y: 30, opacity: 0, stagger: 0.12 }, '-=0.5')
```

```
16 |         .from('.hero-stat', { x: 40, opacity: 0, stagger: 0.12 }, '<')
17 |
18 |       gsap.to('.hero-content', {
19 |         opacity: 0,
20 |         scale: 0.94,
21 |         y: -60,
22 |         ease: 'none',
23 |         scrollTrigger: {
24 |           trigger: containerRef.current,
25 |           start: 'bottom 75%',
26 |           end: 'bottom 15%',
27 |           scrub: true,
28 |         },
29 |       })
30 |     },
31 |     { scope: containerRef, dependencies: [ready] },
32 |   )
33 |
34 |   return (
35 |     <section id="banner" ref={containerRef} className="relative overflow-hidden">
36 |       <div className="hero-content app-container flex min-h-svh flex-col justify-center gap-14
   |         py-24 lg:flex-row lg:items-center lg:justify-between">
37 |         <div className="max-w-xl">
38 |           <h1 className="font-anton text-6xl leading-[1.05] sm:text-7xl md:text-8xl">
39 |             <span className="block overflow-hidden">
40 |               <span className="hero-title-line block text-primary">{HERO.titleTop}</span>
41 |             </span>
42 |             <span className="block overflow-hidden">
43 |               <span className="hero-title-line block sm:ml-2">{HERO.titleBottom}</span>
44 |             </span>
45 |           </h1>
46 |
47 |           <p className="hero-fade mt-6 text-base text-muted-foreground sm:text-lg">
48 |             Hi! I&apos;m <span className="font-semibold
   |               text-foreground">{GENERAL_INFO.name}</span>.{' '}
49 |             {HERO.intro}
50 |           </p>
51 |
52 |           <div className="hero-fade mt-9">
53 |             <Button href={`mailto:${GENERAL_INFO.email}`}>Let&apos;s Talk</Button>
54 |           </div>
55 |
56 |           <p className="hero-fade mt-4 flex items-center gap-2 text-sm text-muted-foreground">
57 |             <span className="relative flex size-2.5">
58 |               <span className="absolute inline-flex h-full w-full animate-ping rounded-full
   |                 bg-primary opacity-75" />
59 |               <span className="relative inline-flex size-2.5 rounded-full bg-primary" />
60 |             </span>
61 |             {HERO.availability}
62 |           </p>
63 |         </div>
64 |
65 |         <div className="grid grid-cols-3 gap-4 sm:gap-8 lg:flex lg:flex-col lg:gap-10
   |           lg:text-right">
66 |           {STATS.map((stat) => (
67 |             <div key={stat.label} className="hero-stat">
68 |               <h5 className="mb-1 font-anton text-3xl text-primary sm:text-4xl">{stat.value}</h5>
69 |               <p className="text-xs text-muted-foreground sm:text-sm">{stat.label}</p>
70 |             </div>
71 |           ))}
72 |         </div>
73 |       </div>
74 |     </section>
75 |   )
76 | }
```

src/sections/AboutMe.jsx

The statement is split into words, each wrapped in a span with class `about-word`. A scrubbed ScrollTrigger animates them from 15% to 100% opacity with a stagger, so the sentence lights up word by word in sync with the scroll position.

Below it, "This is me.", a divider, the greeting and the two bio paragraphs fade up once when they enter the viewport.

```
import { useGSAP } from '@gsap/react'
import gsap from 'gsap'
import { useRef } from 'react'
import { ABOUT } from '../data/portfolio'

export default function AboutMe() {
  const containerRef = useRef(null)

  useGSAP(
    () => {
      gsap.fromTo(
        '.about-word',
        { opacity: 0.15 },
        {
          opacity: 1,
          stagger: 0.1,
          ease: 'none',
          scrollTrigger: {
            trigger: '.about-statement',
            start: 'top 85%',
            end: 'bottom 45%',
            scrub: true,
          },
        },
      )

      gsap.from('.about-reveal', {
        y: 40,
        opacity: 0,
        stagger: 0.15,
        duration: 0.9,
        ease: 'power3.out',
        scrollTrigger: { trigger: '.about-body', start: 'top 80%' },
      })
    },
    { scope: containerRef },
  )

  return (
    <section id="about-me" ref={containerRef} className="py-20 md:py-32">
      <div className="app-container">
        <h2 className="about-statement text-3xl font-thin leading-tight sm:text-4xl md:text-5xl
          lg:text-6xl">
          {ABOUT.statement.split(' ').map((word, i) => (
            <span key={i} className="about-word">
              {word}{' '}
            </span>
          ))}
        </h2>

        <div className="about-body mt-20 md:mt-32">
          <p className="about-reveal pb-3 text-sm text-muted-foreground">This is me.</p>
          <div className="about-reveal border-t border-border" />
          <div className="mt-8 grid gap-8 md:grid-cols-12">
            <h3 className="about-reveal text-3xl sm:text-4xl md:col-span-5">{ABOUT.greeting}</h3>
            <div className="space-y-4 text-muted-foreground md:col-span-6 md:col-start-7
              lg:col-span-5 lg:col-start-7">
              {ABOUT.paragraphs.map((p) => (
                <p key={p.slice(0, 20)} className="about-reveal leading-relaxed">
                  {p}
                </p>
```

```
              ))}
            </div>
          </div>
        </div>
      </div>
    </section>
  )
}
```

src/sections/Skills.jsx

Iterates over `Object.entries(MY_STACK)` to render one row per category. Each row gets its own ScrollTrigger so the category name and icons stagger in as that row scrolls into view. Icons are React components received from the data file and coloured with their brand colour; on hover they lift and scale slightly.

```
import { useGSAP } from '@gsap/react'
import gsap from 'gsap'
import { useRef } from 'react'
import SectionTitle from '../components/ui/SectionTitle'
import { MY_STACK } from '../data/portfolio'

export default function Skills() {
  const containerRef = useRef(null)

  useGSAP(
    () => {
      gsap.utils.toArray('.stack-row').forEach((row) => {
        gsap.from(row.querySelectorAll('.stack-anim'), {
          y: 40,
          opacity: 0,
          stagger: 0.06,
          duration: 0.7,
          ease: 'power3.out',
          scrollTrigger: { trigger: row, start: 'top 85%' },
        })
      })
    },
    { scope: containerRef },
  )

  return (
    <section id="my-stack" ref={containerRef} className="py-20 md:py-32">
      <div className="app-container">
        <SectionTitle title="My Stack" />

        <div className="space-y-16 md:space-y-20">
          {Object.entries(MY_STACK).map(([category, items]) => (
            <div key={category} className="stack-row grid gap-6 sm:grid-cols-12">
              <div className="sm:col-span-5">
                <p className="stack-anim font-anton text-4xl uppercase leading-none
                  text-muted-foreground sm:text-5xl">
                  {category}
                </p>
              </div>
              <ul className="flex flex-wrap gap-x-10 gap-y-6 sm:col-span-7">
                {items.map(({ name, icon: Icon, color }) => (
                  <li key={name} className="stack-anim group flex items-center gap-3">
                    <Icon
                      size={38}
                      style={{ color }}
                      className="transition-transform duration-300 group-hover:-translate-y-1
                        group-hover:scale-110"
                      aria-hidden="true"
                    />
                    <span className="text-lg sm:text-xl">{name}</span>
                  </li>
                ))}
              </ul>
```

```
            </div>
          ))}
        </div>
      </div>
    </section>
  )
}
```

src/sections/Projects.jsx

Renders each project as a React Router `Link` to its detail page, with a two-digit index, title in Anton, subtitle and tech stack separated by dots.

Cursor-following preview (desktop): `gsap.quickTo` creates two optimised setter functions for the preview's x and y. On mouse move the cursor position is converted to coordinates relative to the list and clamped so the card never overflows the right edge. `activeIndex` decides which project's cover is visible and whether the preview is shown at all (with a scale and fade transition).

On phones, where there is no hover, each project shows its cover inline instead (`md:hidden`).

```
import { useGSAP } from '@gsap/react'
import gsap from 'gsap'
import { useRef, useState } from 'react'
import { Link } from 'react-router-dom'
import ProjectThumbnail from '../components/ui/ProjectThumbnail'
import SectionTitle from '../components/ui/SectionTitle'
import { PROJECTS } from '../data/portfolio'

export default function Projects() {
  const containerRef = useRef(null)
  const previewRef = useRef(null)
  const moveRef = useRef(null)
  const [activeIndex, setActiveIndex] = useState(null)

  useGSAP(
    () => {
      gsap.from('.project-item', {
        y: 60,
        opacity: 0,
        stagger: 0.15,
        duration: 0.9,
        ease: 'power3.out',
        scrollTrigger: { trigger: '.project-list', start: 'top 80%' },
      })

      moveRef.current = {
        x: gsap.quickTo(previewRef.current, 'x', { duration: 0.5, ease: 'power3' }),
        y: gsap.quickTo(previewRef.current, 'y', { duration: 0.5, ease: 'power3' }),
      }
    },
    { scope: containerRef },
  )

  const handleMouseMove = (e) => {
    if (!moveRef.current) return
    const rect = containerRef.current.querySelector('.project-list').getBoundingClientRect()
    const previewWidth = previewRef.current.offsetWidth
    moveRef.current.x(Math.min(e.clientX - rect.left + 30, rect.width - previewWidth))
    moveRef.current.y(e.clientY - rect.top - 120)
  }

  return (
    <section id="selected-projects" ref={containerRef} className="py-20 md:py-32">
      <div className="app-container">
        <SectionTitle title="Projects" />

```

```
47 |         <div
48 |           className="project-list relative"
49 |           onMouseMove={handleMouseMove}
50 |           onMouseLeave={() => setActiveIndex(null)}
51 |         >
52 |           {PROJECTS.map((project, index) => (
53 |             <Link
54 |               key={project.slug}
55 |               to={`/projects/${project.slug}`}
56 |               onMouseEnter={() => setActiveIndex(index)}
57 |               className="project-item group block border-b border-border py-6 md:py-8"
58 |             >
59 |               <div className="mb-5 aspect-video overflow-hidden md:hidden">
60 |                 <ProjectThumbnail project={project} />
61 |               </div>
62 |
63 |               <div className="flex gap-3 sm:gap-5">
64 |                 <span className="mt-1 font-anton text-sm text-muted-foreground sm:text-base">
65 |                   _{String(index + 1).padStart(2, '0')}.
66 |                 </span>
67 |                 <div className="min-w-0">
68 |                   <h4 className="font-anton text-4xl leading-tight transition-colors
   |                     duration-300 group-hover:text-primary sm:text-5xl md:text-6xl">
69 |                     {project.title}
70 |                   </h4>
71 |                   <p className="mt-1 text-sm text-muted-foreground
   |                     sm:text-base">{project.subtitle}</p>
72 |                   <ul className="mt-3 flex flex-wrap items-center gap-x-3 gap-y-1 text-xs
   |                     text-muted-foreground sm:text-sm">
73 |                     {project.techStack.map((tech, i) => (
74 |                       <li key={tech} className="flex items-center gap-3">
75 |                         {i > 0 && <span className="size-1 rounded-full bg-background-light" />}
76 |                         {tech}
77 |                       </li>
78 |                     ))}
79 |                   </ul>
80 |                 </div>
81 |               </div>
82 |             </Link>
83 |           ))}
84 |
85 |           {/* Hover preview following the cursor (desktop only) */}
86 |           <div
87 |             ref={previewRef}
88 |             className={`pointer-events-none absolute left-0 top-0 z-10 hidden h-[240px]
   |               w-[400px] overflow-hidden shadow-2xl transition-[opacity,scale] duration-300
   |               md:block ${
89 |               activeIndex === null ? 'scale-75 opacity-0' : 'scale-100 opacity-100'
90 |             }`}
91 |           >
92 |             {PROJECTS.map((project, index) => (
93 |               <div
94 |                 key={project.slug}
95 |                 className={`absolute inset-0 transition-opacity duration-300 ${
96 |                   activeIndex === index ? 'opacity-100' : 'opacity-0'
97 |                 }`}
98 |               >
99 |                 <ProjectThumbnail project={project} />
100 |               </div>
101 |             ))}
102 |           </div>
103 |         </div>
104 |       </div>
105 |     </section>
106 |   )
107 | }
```

Pages

src/pages/Home.jsx

Composes the four sections in order. If the page was opened with a hash (for example after choosing "Projects" in the menu from a project page), it waits until `ready`, smooth-scrolls to that section, then clears the hash from the URL while keeping React Router's history state intact.

```
import { useEffect } from 'react'
import { useLocation } from 'react-router-dom'
import { scrollToSection } from '../hooks/useSmoothScroll'
import AboutMe from '../sections/AboutMe'
import Hero from '../sections/Hero'
import Projects from '../sections/Projects'
import Skills from '../sections/Skills'

export default function Home({ ready }) {
  const { hash } = useLocation()

  useEffect(() => {
    if (!ready || !hash) return
    const timer = setTimeout(() => {
      scrollToSection(hash.slice(1))
      window.history.replaceState(window.history.state, '', window.location.pathname)
    }, 200)
    return () => clearTimeout(timer)
  }, [ready, hash])

  return (
    <>
      <Hero ready={ready} />
      <AboutMe />
      <Skills />
      <Projects />
    </>
  )
}
```

src/pages/ProjectDetails.jsx

Reads `slug` from the URL with `useParams` and finds the matching project. An unknown slug redirects home with `<Navigate>`.

Content fades up in sequence and the cover is revealed with an animated `clip-path` wipe. The animation depends on `slug`, so it replays when moving between projects.

Live Preview and Source Code buttons appear only when those URLs are filled in. A large "Next project" link at the bottom wraps around to the first project using the modulo operator.

```
import { useGSAP } from '@gsap/react'
import gsap from 'gsap'
import { useRef } from 'react'
import { FiArrowLeft, FiArrowRight, FiExternalLink, FiGithub } from 'react-icons/fi'
import { Link, Navigate, useParams } from 'react-router-dom'
import Button from '../components/ui/Button'
import ProjectThumbnail from '../components/ui/ProjectThumbnail'
import { PROJECTS } from '../data/portfolio'

export default function ProjectDetails() {
  const { slug } = useParams()
  const containerRef = useRef(null)
  const index = PROJECTS.findIndex((p) => p.slug === slug)
  const project = PROJECTS[index]
```

```
15 |
16 |   useGSAP(
17 |     () => {
18 |       if (!project) return
19 |       gsap.from('.detail-fade', {
20 |         y: 40,
21 |         opacity: 0,
22 |         stagger: 0.1,
23 |         duration: 0.8,
24 |         ease: 'power3.out',
25 |       })
26 |       gsap.from('.detail-cover', {
27 |         clipPath: 'inset(100% 0 0 0)',
28 |         duration: 1.2,
29 |         ease: 'power4.out',
30 |         delay: 0.3,
31 |       })
32 |     },
33 |     { scope: containerRef, dependencies: [slug] },
34 |   )
35 |
36 |   if (!project) return <Navigate to="/" replace />
37 |
38 |   const nextProject = PROJECTS[(index + 1) % PROJECTS.length]
39 |
40 |   return (
41 |     <section ref={containerRef} className="pt-28 pb-16 md:pt-36">
42 |       <div className="app-container">
43 |         <Link
44 |           to="/#selected-projects"
45 |           className="detail-fade group mb-10 inline-flex items-center gap-2 text-sm uppercase
   |             tracking-widest text-muted-foreground transition-colors hover:text-primary"
46 |         >
47 |           <FiArrowLeft className="transition-transform group-hover:-translate-x-1" />
48 |           Back
49 |         </Link>
50 |
51 |         <p className="detail-fade text-sm text-muted-foreground">{project.subtitle}</p>
52 |         <h1 className="detail-fade mt-2 font-anton text-5xl leading-none sm:text-7xl md:text-8xl">
53 |           {project.title}
54 |         </h1>
55 |
56 |         <div className="detail-fade mt-8 flex flex-wrap gap-2">
57 |           {project.techStack.map((tech) => (
58 |             <span
59 |               key={tech}
60 |               className="border border-border bg-background-light/60 px-3 py-1 text-xs
   |                 text-muted-foreground sm:text-sm"
61 |             >
62 |               {tech}
63 |             </span>
64 |           ))}
65 |         </div>
66 |
67 |         {(project.liveUrl || project.sourceUrl) && (
68 |           <div className="detail-fade mt-8 flex flex-wrap gap-4">
69 |             {project.liveUrl && (
70 |               <Button href={project.liveUrl} target="_blank" rel="noreferrer">
71 |                 Live Preview <FiExternalLink />
72 |               </Button>
73 |             )}
74 |             {project.sourceUrl && (
75 |               <Button href={project.sourceUrl} target="_blank" rel="noreferrer" variant="outline">
76 |                 Source Code <FiGithub />
77 |               </Button>
78 |             )}
79 |           </div>
80 |         )}
81 |
82 |         <div className="detail-cover mt-12 aspect-video overflow-hidden md:aspect-[21/9]">
83 |           <ProjectThumbnail project={project} />
84 |         </div>
85 |
```

```
          <div className="mt-16 grid gap-10 md:grid-cols-12">
            <div className="md:col-span-4">
              <h2 className="detail-fade font-anton text-3xl uppercase
                text-muted-foreground">Overview</h2>
              <p className="detail-fade mt-2 text-sm text-muted-foreground">Year: {project.year}</p>
            </div>
            <div className="md:col-span-8">
              <p className="detail-fade text-lg leading-relaxed
                sm:text-xl">{project.description}</p>
              <h3 className="detail-fade mt-10 font-anton text-2xl uppercase text-muted-foreground">
                Key Highlights
              </h3>
              <ul className="mt-4 space-y-4">
                {project.highlights.map((item) => (
                  <li key={item} className="detail-fade flex gap-3 leading-relaxed
                    text-muted-foreground">
                    <span className="mt-2.5 size-1.5 shrink-0 rounded-full bg-primary" />
                    {item}
                  </li>
                ))}
              </ul>
            </div>
          </div>

          {nextProject.slug !== project.slug && (
            <Link
              to={`/projects/${nextProject.slug}`}
              className="group mt-24 flex items-center justify-between gap-6 border-t
                border-border pt-10"
            >
              <div>
                <p className="text-sm uppercase tracking-widest text-muted-foreground">Next
                  project</p>
                <p className="mt-2 font-anton text-4xl transition-colors group-hover:text-primary
                  sm:text-6xl">
                  {nextProject.title}
                </p>
              </div>
              <FiArrowRight className="size-10 shrink-0 transition-transform
                group-hover:translate-x-2 group-hover:text-primary" />
            </Link>
          )}
        </div>
      </section>
    )
}
```

src/pages/NotFound.jsx

A simple 404 screen with a large green "404" and a button back to the home page.

```
import Button from '../components/ui/Button'

export default function NotFound() {
  return (
    <section className="app-container flex min-h-svh flex-col items-center justify-center
      text-center">
      <h1 className="font-anton text-8xl text-primary sm:text-9xl">404</h1>
      <p className="mt-4 text-lg text-muted-foreground">This page drifted off into space.</p>
      <Button as="link" to="/" className="mt-10">
        Go Home
      </Button>
    </section>
  )
}
```

9. Customising and Deploying

Updating content

To change...	Edit this in src/data/portfolio.js
Name, email, contact message	GENERAL_INFO
GitHub / LinkedIn links	SOCIAL_LINKS (replace the "#" placeholders with real URLs)
Hero headline and intro	HERO
About statement and bio	ABOUT
Skills and icons	MY_STACK (import extra icons from react-icons/si)
Stats under the hero	STATS
Projects	PROJECTS (add an object; the list and its detail page are created automatically)
Project screenshot	Put the image in public/projects/ and set image: "/projects/name.png"
Live demo / GitHub buttons	Set liveUrl and sourceUrl on the project

Changing the look

Edit the colour values in the `@theme` block of `src/index.css`. Because every component uses the token classes (`text-primary`, `bg-background` and so on), the whole site updates at once.

Building and deploying

```
npm run build      # outputs static files to dist/
npm run preview    # test the build locally
```

The `dist/` folder is a static site and can be hosted on Vercel, Netlify, GitHub Pages or any static host. Because the site uses client-side routing, the host must serve `index.html` for every path so that project URLs work when opened directly. On Netlify, add a `public/_redirects` file containing `/* /index.html 200`; on Vercel add a rewrite in `vercel.json`.

10. Testing and Verification

- **Build:** `npm run build` completes without errors after every change.
- **Lint:** oxlint reports no warnings over `src/`.
- **Visual checks:** the site was opened in a browser at 1440px desktop and 390px phone widths to confirm the hero, menu, stack, projects list and detail pages lay out correctly.
- **Menu:** opening, closing with Escape, and the scroll lock were tested.
- **Reload behaviour:** scrolling to Projects then reloading, and opening a URL ending in `#selected-projects`, both start at the top of the home page.

- **Routing:** project detail routes render the correct content, and unknown slugs redirect home.